

Harshavardhini Jonnalagadda

California, USA | 747-836-6289

<https://hars426.github.io/portfolio-website>

mail to: harshavardhinijonnalagadda03@gmail.com | www.linkedin.com/in/harshavardhini-j

SUMMARY

Design Verification Engineer with hands-on experience in SystemVerilog, UVM, and RTL verification, building testbenches, debugging designs, and ensuring functional coverage.

Comfortable using Python and TCL for automation, with a strong foundation in computer architecture and verification concepts.

TECHNICAL SKILLS

Languages & HDLs: Verilog, SystemVerilog, VHDL, C, C++, Python, TCL, perl.

Verification: UVM, Assertion-Based Verification, Functional Coverage, Constrained-Random Verification.

Formal Verification & Concepts: Basic Formal Logic, Intro to Model Checking, Intro to Static Analysis.

Protocols: AXI, AHB, APB, UART, SPI, I2C.

Tools: Synopsys VCS, Verdi, ModelSim, Vivado.

Concepts: RTL Design, FSM, Pipelining, Debugging, SRAM, DRAM, OOPS, SVA.

Environment: Linux.

EDUCATION

M.S. in Computer Engineering, California State University, Northridge, **Aug 2026**

B.Tech in Computer Engineering, Mahatma Gandhi University, **2023**

EXPERIENCE

Design Verification Trainee (Hands-on Industry Training), Sumedha IT Solutions (Dec 2022 - Dec 2023)

1. Developed **SystemVerilog testbenches** for RTL verification with focus on functional correctness and edge-case validation.
 2. Applied **UVM methodology** using driver, monitor, and scoreboard for structured verification.
 3. Performed **RTL debugging using waveform analysis** to identify and resolve functional mismatches.
 4. Validated designs using **AXI/APB protocol concepts and assertion-based checks**
 5. Used **Synopsys VCS and Verdi** for simulation, debugging, and waveform analysis
 6. Automated workflows using **Python and TCL scripting** in a Linux environment.
 7. Explored **basic formal verification concepts** and collaborated with team members to improve verification quality.
 8. Applied knowledge of **CPU architecture concepts** including pipelining, hazard detection, and control flow during design verification tasks
-

PROJECTS

Parameterized ALU + Register File Datapath

1. Built this project to understand how **RTL datapaths and processor-style computation** work internally.
 2. Designed a **parameterized 32-bit ALU, register file, datapath integration**, and verified functionality through simulations and waveform debugging.
 3. Faced challenges while **connecting datapath modules and debugging incorrect ALU outputs**, which were resolved through waveform analysis.
 4. Successfully achieved **stable arithmetic and logical operations with correct datapath execution**.
 5. Tools used: Verilog, SystemVerilog, Icarus Verilog, GTKWave, Git/GitHub.
-

Synchronous FIFO Design & Verification

1. Built this project to understand **FIFO memory architecture and verification methodologies** used in RTL design.
 2. Designed a parameterized synchronous FIFO with **overflow/underflow protection, SVA assertions**, and interface-based SystemVerilog verification.
 3. Faced issues **with pointer handling and FIFO boundary conditions**, which were resolved through assertions and waveform debugging.
 4. Successfully verified stable **FIFO read/write behavior with correct status-flag operation**.
 5. Tools used: Verilog, SystemVerilog, SVA, GTKWave, Icarus Verilog, Git/GitHub.
-

COURSEWORK: Digital Design, Computer Architecture, VLSI Design, ASIC Development, System on Chip (SoC), Advanced Switching Theory, System Verilog, Digital System Structure, Machine Learning, Database Design.